

CS69021: Computing Lab-1

Practice Assignment || August 12, 2026

Topic: Heap & Trees

Instructions

- Read the problem. Type it yourself, from scratch, without any AI help. Get it wrong. Debug it. That struggle is literally the only part of this that builds the skill you're being tested on.
- Always check the constraints before coding. Use them to figure out what time and space complexity will actually pass.
- Submit whatever you have done by 5 PM on Moodle in a single zip <rollno>_wk4_prc.zip.

Do the rest at home.

Part A — Heap

Q1. Minimum Cost to Connect Ropes

Problem Link: <https://www.geeksforgeeks.org/problems/minimum-cost-of-ropes-1587115620/1>

Given an array `arr[]` of rope lengths, connect all ropes into a single rope with the minimum total cost. The cost to connect two ropes is the sum of their lengths. Return the minimum possible cost.

Examples

Input	Output	Explanation
<code>arr[] = [4, 3, 2, 6]</code>	29	First connect 2 and 3 → [4, 5, 6], cost 5. Then connect 4 and 5 → [9, 6], cost 9. Finally connect 9 and 6 → cost 15. Total = 5 + 9 + 15 = 29. Any other order (e.g. connecting 4 and 6 first) gives a higher total (38).
<code>arr[] = [4, 2, 7, 6, 9]</code>	62	Connect 4 & 2 → [6, 7, 6, 9], cost 6. Connect 6 & 6 → [12, 7, 9], cost 12. Connect 7 & 9 → [12, 16], cost 16. Connect 12 & 16 → cost 28. Total = 6 + 12 + 16 + 28 = 62.

Q2. Maximum Sum Combination

Problem Link: <https://www.geeksforgeeks.org/problems/maximum-sum-combination/1>

You are given two integer arrays `a[]` and `b[]` of equal size. A sum combination is formed by adding one element from `a[]` and one from `b[]`, using each index pair (i, j) at most once. Return the top `k` maximum sum combinations, sorted in non-increasing order.

Examples

Input	Output	Explanation
<code>a[] = [3, 2], b[] = [1, 4], k = 2</code>	[7, 6]	Possible sums: 3+1=4, 3+4=7, 2+1=3, 2+4=6. Top 2 sums are 7 and 6.
<code>a[] = [1, 4, 2, 3], b[] = [2, 5, 1, 6], k = 3</code>	[10, 9, 9]	The top 3 maximum possible sums are 4+6=10, 3+6=9, and 4+5=9.

Q3. Find Median from Data Stream

Problem Link: <https://leetcode.com/problems/find-median-from-data-stream/>

You are given a sequence of integers arriving one by one as a data stream. After each insertion, efficiently determine the median of all numbers received so far.

- If the total number of elements is odd, the median is the middle element after sorting.
- If the total number of elements is even, the median is the average of the two middle elements.

Task — process a series of queries of two types

- Insert a number into the data stream.
- Print the current median of all inserted numbers.

Input

- The first line contains an integer Q , the number of queries.
- Each of the next Q lines is one of the following:
 - 1 X — Insert the integer X into the data stream.
 - 2 — Print the median of all numbers inserted so far.

Output

- For every query of type 2, print the median on a new line.
- If the number of elements is even, print the median as a floating-point number.

Constraints

- $1 \leq Q \leq 2 \times 10^5$
- $-10^9 \leq X \leq 10^9$
- It is guaranteed that a query of type 2 is issued only after at least one number has been inserted.

Example

Input	Output	Explanation
8 1 5 1 3 2 1 8 2 1 2 1 10 2	 4.0 5.0 5.0 	 Insert 5, 3 → sorted [3,5] → median $(3+5)/2 = 4.0$. Insert 8 → [3,5,8] → median 5.0. Insert 2, 10 → [2,3,5,8,10] → median 5.0.

Part B — Tree

Q4. Tree Traversals

Given a binary tree, write a program to perform and print the following traversals:

- Inorder Traversal (Left → Root → Right)
- Preorder Traversal (Root → Left → Right)
- Postorder Traversal (Left → Right → Root)
- Level Order Traversal (Breadth-First Search, level by level)
- Boundary Traversal, consisting of:
 - The root node.
 - The left boundary (excluding leaf nodes).
 - All leaf nodes from left to right.
 - The right boundary (excluding leaf nodes) in bottom-up order.

Input

- The first line contains an integer N, the number of inputs.
- The next line takes n input

Note : -1001 if the node is NULL.

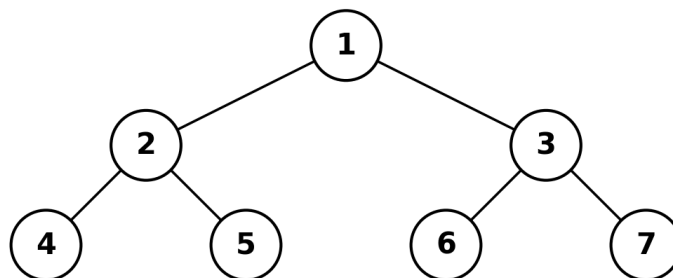
Output

Print the traversals in the format: Inorder / Preorder / Postorder / Level Order / Boundary, each followed by space-separated node values.

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{Node Value} \leq 10^9$
- All node values are unique.

Example



Input	Output	Explanation
N=7 Root =[1,2,3,4,5,6,7]	Inorder: 4 2 5 1 6 3 7 Preorder: 1 2 4 5 3 6 7 Postorder: 4 5 2 6 7 3 1 Level Order: 1 2 3 4 5 6 7	Standard binary tree built from the given parent/child triples; each traversal is computed by the rule stated above.

Input	Output	Explanation
	Boundary: 1 2 4 5 6 7 3	

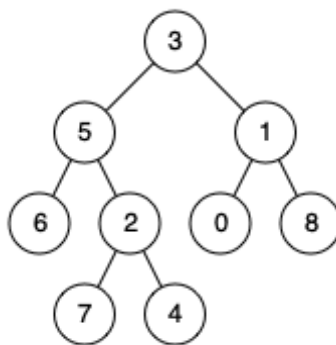
Q5. Lowest Common Ancestor (LCA) of a Tree

Problem Link: <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/description/>

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. Per the Wikipedia definition: the LCA of two nodes p and q is the lowest node in T that has both p and q as descendants (a node is allowed to be a descendant of itself).

NOTE : -1001 denote NULL in i/p case

Example 1



Input	Output	Explanation
n=11 root = [3,5,1,6,2,0,8,-1,-1,7,4] , p = 5, q = 1	3	The LCA of nodes 5 and 1 is 3.

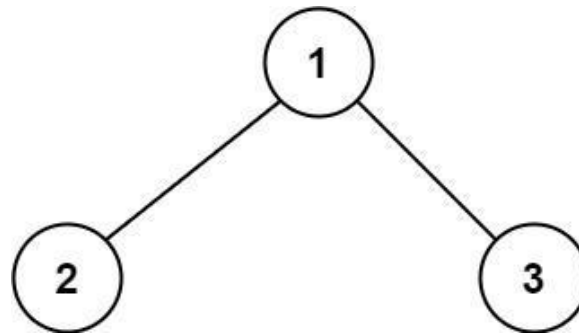
Q6. Maximum Path Sum

Problem Link: <https://leetcode.com/problems/binary-tree-maximum-path-sum/description/>

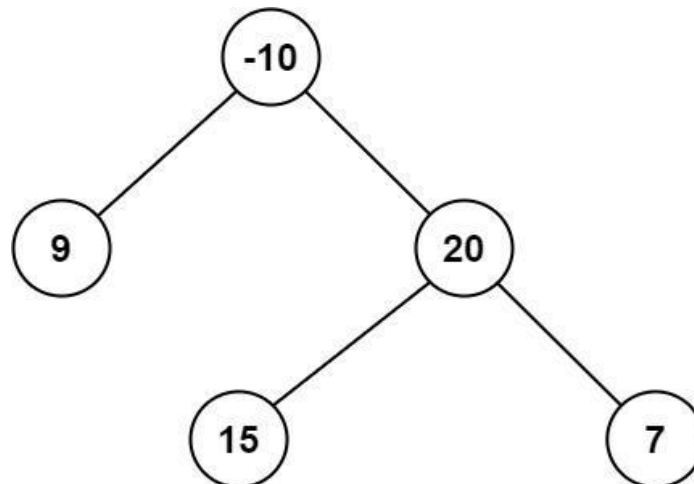
A path in a binary tree is a sequence of nodes where each pair of adjacent nodes has an edge connecting them, and a node can appear at most once. The path need not pass through the root. The path sum is the sum of the nodes' values in the path. Given the root of a binary tree, return the maximum path sum of any non-empty path.

Note : -1001 denote NULL input in ip case

Example 1



Example 2



Input	Output	Explanation
N=3 root = [1,2,3]	6	The optimal path is 2 → 1 → 3, with a path sum of 2 + 1 + 3 = 6.
N=7 root = [-10,9,20,-1001,-1001,15,7]	42	The optimal path is 15 → 20 → 7, with a path sum of 15 + 20 + 7 = 42.

Constraints

- Number of nodes is in the range $[1, 3 \times 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

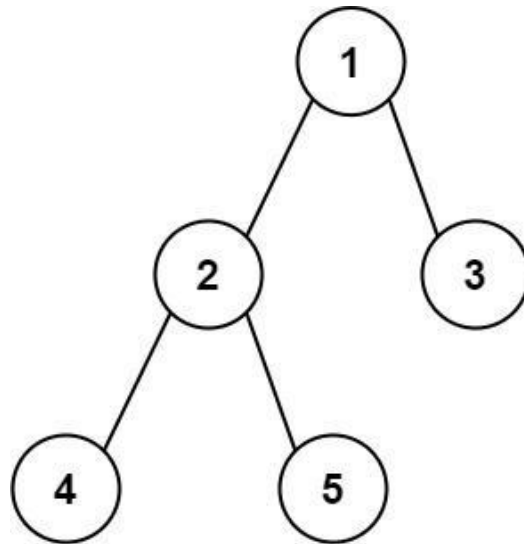
Q7. Diameter of a Binary Tree

Problem Link: <https://leetcode.com/problems/diameter-of-binary-tree/description/>

Given the root of a binary tree, return the length of the diameter of the tree — the length of the longest path between any two nodes (may or may not pass through the root). The length of a path is the number of edges between the two nodes.

NOTE: -1001 denote NULL VALUES in ip case

Example 1



Input	Output	Explanation
N=5 root = [1,2,3,4,5]	3	3 is the length of the path [4,2,1,3] or [5,2,1,3].
N=2 root = [1,2]	1	The single edge between nodes 1 and 2 gives a diameter of 1.

Constraints

- Number of nodes is in the range $[1, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

Part C — Binary Search Tree

Q8. Delete Node in a BST

Problem Link: <https://leetcode.com/problems/delete-node-in-a-bst/>

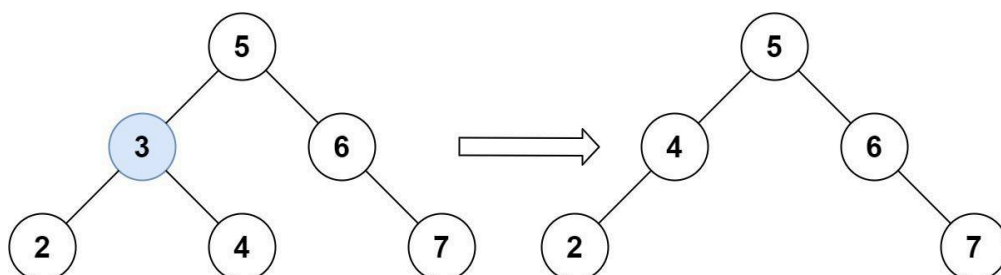
Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

The deletion is divided into two stages

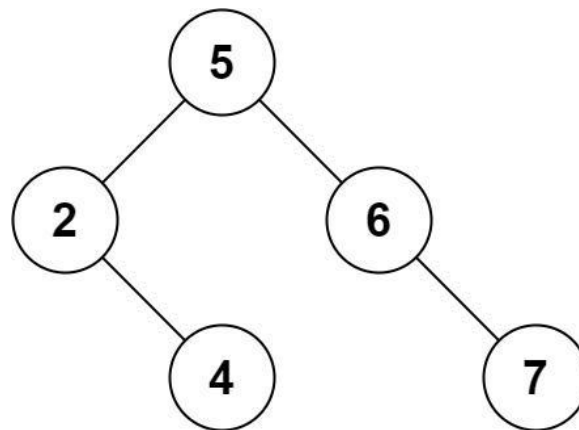
- Search for a node to remove.
- If the node is found, delete the node.

Note : -1001 denote null in ip case

Example 1



Example 2 & 3



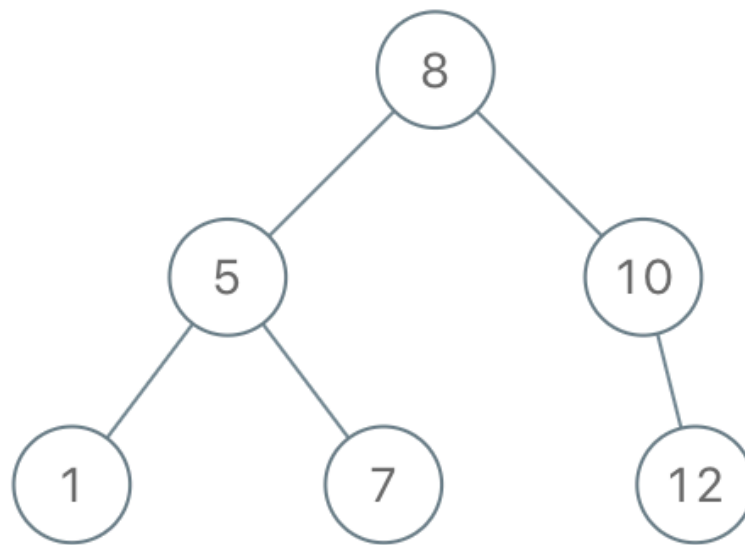
Input	Output	Explanation
N=7 root = [5,3,6,2,4,-1001,7], key = 3	[5,4,6,2,null,null,7]	Node with value 3 is found and deleted. Another valid answer is [5,2,6,null,4,null,7] — both are accepted.
N=7 root = [5,3,6,2,4,-1001,7], key = 0	[5,3,6,2,4,null,7]	The tree does not contain a node with value 0, so it is returned unchanged.
N=0 root = [], key = 0	[]	An empty tree remains empty.

Q9. Construct BST from Preorder Traversal

Problem Link: <https://leetcode.com/problems/construct-binary-search-tree-from-preorder-traversal/description/>

Given an array preorder representing the preorder traversal of a BST, construct the tree and return its root. It is guaranteed that a valid BST exists for each test case. A BST is a binary tree where every node's left subtree contains only values strictly less than the node's value, and its right subtree contains only values strictly greater. A preorder traversal visits the node itself first, then its left subtree, then its right subtree.

Example 1



Input	Output	Explanation
N=6 preorder = [8,5,1,7,10,12]	[8,5,10,1,7,null,12]	Values less than 8 (5,1,7) form the left subtree; values greater than 8 (10,12) form the right subtree, recursively.
N=2 preorder = [1,3]	[1,null,3]	3 > 1, so it becomes the right child of 1.

Constraints

- $1 \leq \text{preorder.length} \leq 100$
- $1 \leq \text{preorder}[i] \leq 1000$
- All values of preorder are unique.